

Chapter 13: NP-completeness

13.1 Problems in P

We consider “decision problems” for graphs. The input is a graph and maybe some other parameters (such as an integer k), collectively known as an “instance”, and there is a single yes/no question. Like: “Does the graph have an Eulerian circuit?” or “Does the graph have a clique of size k ?”

An *algorithm* is a set of clear instructions for solving a problem: the instructions break down into a sequence of steps. For an algorithm for a graph problem, let $f(n)$ denote the maximum number of steps that the algorithm would take when confronted with a graph on n vertices. The function $f(n)$ is known as the *running time* of the algorithm. Such an algorithm is said to be *polynomial-time* if there exists a polynomial $g(n)$ such that $f(n) \leq g(n)$ for all n . The set of all problems that can be solved in polynomial time is denoted by **P**. Colloquially people say such problems have “efficient” algorithms.

For example, consider the problem to determine if an input connected graph has an Eulerian circuit or not. Then a reasonable algorithm would simply calculate for each vertex the degree of that vertex and check that all degrees are even. Assume we are given the graph as an adjacency matrix. (The exact representation does not matter when we define P.) Then checking the parity of the degree of a particular vertex takes at most n steps. And the whole algorithm takes about n^2 steps. Thus there is an efficient algorithm to test whether a connected graph has an Eulerian circuit or not.

Common graph decision problems in P include:

- Is it bipartite?
- Is there a perfect matching?
- Is the distance between two specified vertices at most some specified value?
- Is the connectivity at least some specified value?

There are, of course, many graph problems where the answer is a value rather than just yes–no. We do not consider these explicitly, except to note that the

two types of problems often can be related. For example, if one can decide whether the connectivity is at least some threshold, then one can determine the actual connectivity by trying all thresholds (or to be more efficient using a binary search).

13.2 Problems in NP and NP-completeness

There are many graph problems for which efficient algorithms are not known. Suppose I know that the graph has a Hamilton cycle and I wish to convince you that there is one. Well I can simply tell you the Hamilton cycle and then you can easily verify that it is valid. But how would I prove to you that there was not a Hamilton cycle?

An instance of the problem for which the answer is yes is known as a yes-instance. For the problem of

INPUT: a graph. QUESTION: Does the graph have a Hamilton cycle?

there are short *certificates* or “proofs” for yes-instances.

A problem is in **NP** if there exists an efficient algorithm which, given an instance I and a potential certificate C , can tell whether the C gives a valid proof that the answer is yes. It is important to note that: If the instance is a yes-instance, then there must exist a certificate that will convince the algorithm. If the instance is a no-instance, then there must be no certificate that will convince the algorithm. Informally, we say that a problem is in NP if there exists an efficient algorithm which with some prompting can verify a yes-answer.

There are tens of thousands of graph problems in NP. For example,

- Does the graph have a Hamilton cycle?
- Is it 3-colorable?
- Is it decomposable into K_3 's?
- Is the independence number at least k ?

The certificate is, respectively, the cycle, the coloring, the decomposition, and the independent set.

Obviously $P \subseteq NP$. A major open question is whether $P = NP$ or not.

A problem is said to be **NP-complete** if the existence of an efficient algorithm for it would imply the existence of an efficient algorithm for every problem in NP. Essentially, the NP-complete problems are the hardest ones in NP. All four problems listed above are NP-complete. A problem is said to be **NP-hard** if it is at least as hard as all problems in NP.

13.3 Proving a Problem NP-Complete

Assume one has a new problem, say X , that is in NP. To prove that X is NP-complete, one provides a **reduction** from some known NP-complete problem, say K , to X . A reduction transforms a question in the language of K into the language of X without changing the answer. Several examples are provided below.

Note that one has complete freedom in the choice of K . A common choice is called SAT. For this problem the input is a boolean formula (meaning each variable can be either true or false) and the question is whether the formula is **satisfiable**: Is there a setting of the variables that makes the formula true. The formula is given in “conjunctive normal form”: this means it is the AND of a collection of **clauses**, where each clause is the OR of a collection of variables and negated variables, collectively called **literals**. An example formula is $(x \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee w)$. This is easily satisfiable: for example when all variables are true, both clauses are true, and hence the formula is true.

13.3.1 Domination: Proof One

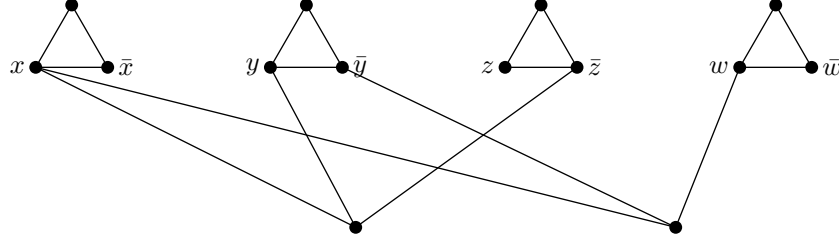
The DOMINATION problem has input a graph G and an integer k , and the answer is whether the graph has a dominating set of size at most k .

THEOREM. *The DOMINATION problem is NP-complete.*

The classical way to show this is to reduce SAT to it. When reducing SAT to a graph problem, it is common to create little subgraphs, called **gadgets**, for each variable, and for each clause, and then to join these up in some fashion.

Here is the reduction to DOMINATION: Consider the boolean formula ϕ . For each variable, say x , construct a triangle with two vertices labeled x and $\bar{x}$. For each clause, add a vertex. Then join each clause-vertex to the vertices representing the literals in that clause. Call the resultant graph G_ϕ .

For example, if $\phi = (x \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee w)$, then graph G_ϕ will look like this:



Claim: ϕ has a satisfying assignment if and only if G_ϕ has a dominating set of size k , where k is the number of variables used in ϕ . (In the example $k = 4$.)

Assume G_ϕ has a dominating set S of size k . Then, since one needs to dominate the unlabeled vertex in each variable-gadget, the set S has exactly one vertex from each variable-gadget, and each clause vertex is dominated by one of the literals. Thus one can create a satisfying assignment for ϕ by setting x to be true if $x \in S$ and x to be false if $\bar{x} \in S$. Conversely, if one has a satisfying assignment, then one can create a dominating set of G_ϕ by adding x to the set if x is true and $\bar{x}$ to the set if x is false.

13.3.2 Domination: Proof Two

Suppose instead that the VERTEXCOVER problem has already been proven to be NP-complete, where the input to that is a graph and a parameter and the question is whether the graph has a vertex cover of at most that size. Then another way to prove that DOMINATION is NP-complete is to reduce VERTEXCOVER to it.

Given a graph G , one can construct graph G' by, for each edge e , adding a new vertex w_e whose neighbors are the ends of e . The claim is that the domination number of G' equals the vertex cover number of G . Any dominating set D of G' must dominate each w_e . We can always choose D to not contain any of the w_e by replacing such a w_e by either of its neighbors. This means we can choose D

to be a vertex cover of G . Conversely, any vertex cover of G is immediately a dominating set of G' .

So the problem of calculating the domination number is at least as hard as the problem of calculating the vertex cover number. Or put in the terminology above, we reduce VERTEXCOVER to DOMINATION by mapping (G, k) to (G', k) .

13.3.3 Defective Coloring

Consider the problem of coloring a graph with two colors, say red and blue, such that every red vertex has no red neighbor, and every blue vertex has at most one blue neighbor. Call this the DEFECTIVE(0,1)-COLORING problem. As usual it is trivial to check that the problem is in NP: the certificate is the actual coloring.

THEOREM. *The DEFECTIVE(0,1)-COLORING problem is NP-complete.*

We will reduce from ONE-IN-THREE SAT. (The book by Garey and Johnson has a cornucopia of NP-complete problems.)

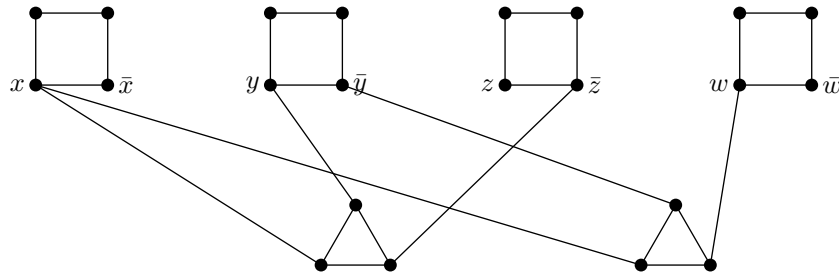
ONE-IN-THREE SAT

Input: Boolean formula ϕ with each clause having three literals

Question: Is there a truth assignment of the variables such that each clause in ϕ has exactly one true literal?

Now for the reduction: Consider the boolean formula ϕ . For each variable, say x , construct a 4-cycle with two consecutive vertices labeled x and $\bar{x}$. For each clause, construct a triangle, and add a matching between the triangle and the vertices corresponding to the literals in the clause. Call the resultant graph G_ϕ .

For example, if $\phi = (x \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee w)$, then graph G_ϕ will look like this:



Claim: ϕ has a one-in-three satisfying assignment if and only if G_ϕ has a defective(0,1)-coloring.

Assume ϕ has a one-in-three satisfying assignment. Color each true literal blue and each false literal red. That is, if x is true in the assignment, then color x blue and $\bar{x}$ false; if x is false in the assignment, then color x red and $\bar{x}$ blue. Complete each 4-cycle alternating colors. Then in each clause triangle, color the vertex the opposite color of its literal. The clause has one blue and two red neighbors. The clause triangle is colored one red and two blues, and every vertex is okay.

Conversely, assume G_ϕ has a defective(0,1)-coloring. Then every variable 4-cycle must be colored alternating red and blue; thus the two literals in it receive opposite colors. And every clause triangle must have one vertex colored red and two colored blue. Further, the red vertex must have a blue neighbor in the vertex gadgets and both blues must have red neighbors in the vertex gadgets (since their quota of blue neighbors is used up). So exactly one of the literals in the clause is blue. Thus the set of blue literals corresponds to a one-in-three assignment of ϕ .

Thus we have shown that ONE-IN-THREE SAT reduces to DEFECTIVE(0,1)-COLORING problem, and so the latter is NP-complete.

13.3.4 Restricted 3-colorability

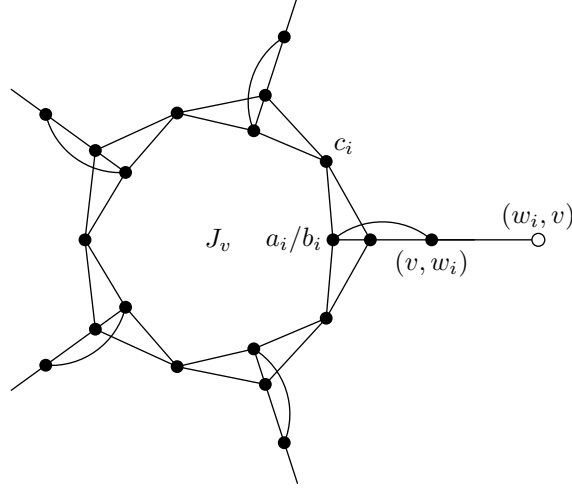
THEOREM. *The 3COLORABILITY problem (given graph G determine if $\chi(G) \leq 3$) is NP-complete even for graphs of maximum degree 4.*

This is a **restriction** of a problem known to be in NP, so it is automatically in NP itself.

We will reduce from 3COLORABILITY. Specifically, given graph G , we will construct graph H_G such that (a) H_G has maximum degree 4, and (b) H_G is 3-colorable if and only if G is 3-colorable.

Now for the reduction. For each vertex v in G , we construct a gadget J_v in H_G as follows. Say the neighbors of v in G are $w_1, w_2, \dots, w_d$. Then introduce vertices (v, w_1) through (v, w_d) . Then for $1 \leq i \leq d$, add vertices a_i, b_i, c_i and add edges $(v, w_i)a_i, (v, w_i)b_i, a_ib_i, a_ic_i, b_ic_i, a_{i+1}c_i$ and $b_{i+1}c_i$. Finally add edges joining

each (v, w_i) to the analogous vertex (w_i, v) in J_{w_i} . The result has maximum degree 4.



Now, suppose G is 3-colorable. Then color H_G as follows. For each vertex v in G , give its color to every vertex of the form (v, w_i) and every c_i in J_v . This can immediately be extended to a proper 3-coloring of H_G by giving the a_i, b_i the two colors other than that of v .

Conversely, suppose H_G is 3-colorable. Then, for each J_v , all the c_i receive the same color and thus so do all the vertices of the form (v, w_i) . We can thus color vertex v of G with the color of (v, w_i) , which is necessarily different to the color of each (w_i, v) .